TypeScript allowSyntheticDefaultImports explanation.

PR Link: https://github.com/kbss-cvut/fta-fmea-ui/pull/153

Conversation Link: https://chat.openai.com/share/3bb73e47-e805-4b43-bc0a-7aa4caf88796

in typescript, what is allowSyntheticDefaultImports?

`allowSyntheticDefaultImports` is an option in TypeScript's compiler settings. When you set `allowSyntheticDefaultImports` to true in your TypeScript configuration (`tsconfig.json`), it allows you to use default imports from modules that do not have a default export.

This option is particularly useful when you're working with modules that are written in JavaScript, where the default export behavior is handled differently compared to TypeScript. In JavaScript, it's common to export modules without a default export, but you might still want to import them as if they had a default export.

For example, consider a JavaScript module that exports an object, function, or class, but not as a default export:

Code

```
// In a JavaScript module
module.exports = function myFunction() {
    // Some functionality
};
```

Without `allowSyntheticDefaultImports` set to true, importing this in TypeScript would typically require using a named import or the `import * as` syntax, like this:

Code

```
// Importing in TypeScript without allowSyntheticDefaultImports
import * as myFunction from 'myModule';
```

However, with `allowSyntheticDefaultImports` set to true, you can import it as if it was a default export:

Code

```
// Importing in TypeScript with allowSyntheticDefaultImports
import myFunction from 'myModule';
```

This setting doesn't change the code that's generated after compiling. It's a type-checking feature that allows for more flexibility and interoperability with JavaScript modules. It's particularly handy when you're working in a codebase that mixes JavaScript and TypeScript or when you're using libraries written in JavaScript that you want to consume in a TypeScript way.

Is it a bad practice to use it?

Using `allowSyntheticDefaultImports` in TypeScript isn't inherently bad practice, but whether or not you should use it depends on your project's context, the interoperability needs with JavaScript libraries, and the build tools you're using.

Considerations for Using `allowSyntheticDefaultImports`:

1. **Interoperability with JavaScript Libraries**: If you're working with a lot of JavaScript libraries that don't have default exports, this option can simplify your import statements and make your TypeScript code look cleaner and more in line with ES6 syntax.

2. **Build Tools Compatibility**: Some bundlers and transpilers (like Webpack and Babel) handle default imports from non-ES6 modules in a way that aligns with `allowSyntheticDefaultImports`. If you're using such tools, enabling this option in TypeScript can make your TypeScript configuration more consistent with how your build tools work.

3. **Codebase Consistency**: If you’re working in a mixed codebase with both JavaScript and TypeScript, or in a team environment, using this option can help standardize module imports across your project, which can improve readability and maintainability.

4. **TypeScript and ES6 Standards**: It's important to note that `allowSyntheticDefaultImports` is a TypeScript-specific feature that helps bridge a gap between TypeScript and the ES6 module system. While it can be useful, it doesn't reflect the actual runtime behavior if you’re targeting ES6 modules

directly.

Potential Issues:

- **Runtime Behavior**: The main risk is misunderstanding the runtime behavior. This setting doesn’t change how the JavaScript is executed; it only affects type checking. If you’re not using a module loader or bundler that automatically handles default imports (like Babel or Webpack), you might run into issues where your code compiles without errors but fails at runtime.

- **Dependency on Build Tools**: Relying heavily on `allowSyntheticDefaultImports` can make your code more dependent on specific build tools and transpilation steps, which could be a consideration if you’re aiming for a build-tool-agnostic setup.

In summary, `allowSyntheticDefaultImports` is a useful feature for certain scenarios, especially when dealing with a lot of JavaScript libraries or aiming for cleaner syntax in a TypeScript project. However, it's crucial to understand how it interacts with your build tools and runtime environment to ensure it doesn't introduce discrepancies between your compiled code and its runtime behavior. As with many features in programming, the key is understanding the context and implications of using it.